American Option Pricing: PSOR vs Operator Splitting

Subash KC

October 28, 2025

Abstract

This report presents a numerical method for pricing American call option using two finite-difference methods: the Projected Successive Over-Relaxation (PSOR) method and the Operator Splitting method proposed by Ikonen and Toivanen (2004). The methods are analyzed in the context of the Linear Complementarity Problem (LCP) formulation of American options. Both methods are implemented and results include the option price against asset price and time. Computation times between both methods is also discussed.

Contents

1 Introduction **2**

2 Mathematical Formulation **2**
2.1 Linear Complementarity Problem (LCP) 2

3 Finite Difference Discretization and PSOR Method **2**
3.1 Finite Difference Discretization 2
3.2 Discrete LCP Formulation 3

4 Numerical Methods **3**
4.1 Projected Successive Over-Relaxation (PSOR) 3
4.2 Operator Splitting Method 3
4.2.1 Mathematical Formulation 4
4.2.2 Algorithm 4

5 Implementation **4**

6 Results **4**

7 Comparision and Discussion **5**

8 Conclusion **5**

9 References **6**

1 Introduction

American options differ from European options by allowing early exercise at any time prior to maturity. American options present a significant computational challenge in quantitative finance due to this early exercise feature. This feature transforms the standard Black-Scholes partial differential equation (PDE) into a free-boundary problem, as the optimal exercise boundary is not known in advance and must be determined simultaneously with the option value. This summary covers the main numerical approaches discussed in Seydel's *Tools for Computational Finance* (Sections 3.6 and 4.6) and in S. Ikonen and J. Toivanen, "Operator Splitting Methods for American Option Pricing," *Applied Mathematics Letters*, vol. 17, no. 7, pp. 809–814, 2004.

2 Mathematical Formulation

The value $V(S,t)$ of an American option satisfies the Black–Scholes PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0,$$

subject to the constraint that $V(S,t) \geq \Phi(S)$, where $\Phi(S)$ is the payoff function. At maturity, $V(S,T) = \Phi(S)$. The early-exercise condition implies:

$$\max\left(\frac{\partial V}{\partial t} + \mathcal{L}_{BS}V,\ \Phi(S) - V\right) = 0,$$

where $\mathcal{L}_{BS}$ is the Black–Scholes differential operator:

$$\mathcal{L}_{BS}V = \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV.$$

2.1 Linear Complementarity Problem (LCP)

Following Seydel, the problem can be formulated as an LCP. For a call option with intrinsic value $\Lambda(S) = \max(S - K, 0)$, the LCP is:

$$\frac{\partial V}{\partial t} + \mathcal{L}_{BS}V \geq 0, \tag{1}$$

$$V(S,t) - \Lambda(S) \geq 0, \tag{2}$$

$$\left(\frac{\partial V}{\partial t} + \mathcal{L}_{BS}V\right)(V(S,t) - \Lambda(S)) = 0. \tag{3}$$

This formulation ensures that either the option satisfies the Black–Scholes PDE (continuation region) or equals its intrinsic value (exercise region), but not both simultaneously.

3 Finite Difference Discretization and PSOR Method

3.1 Finite Difference Discretization

The discretization of the Black–Scholes PDE using the Crank-Nicolson scheme:

$$\frac{V_i^{n+1} - V_i^n}{\Delta t} + \frac{1}{2}\mathcal{L}_{BS}(V_i^n + V_i^{n+1}) = 0, \tag{4}$$

which leads to a tridiagonal system:

$$A\mathbf{V}^{n+1} = \mathbf{b}^n, \tag{5}$$

$$\mathbf{b}^n = B\mathbf{V}^n, \tag{6}$$

where A and B are constructed from finite difference coefficients. The intrinsic value $\Lambda(S)$ imposes the early-exercise constraint.

3.2 Discrete LCP Formulation

Discretizing the PDE in time and space using finite differences transforms it into a discrete LCP at each time step:

$$A\mathbf{V}^{n+1} \geq \mathbf{b}^n, \tag{7}$$

$$\mathbf{V}^{n+1} \geq \mathbf{\Lambda}, \tag{8}$$

$$(\mathbf{V}^{n+1} - \mathbf{\Lambda})^\top (A\mathbf{V}^{n+1} - \mathbf{b}^n) = 0, \tag{9}$$

where A is the system matrix from the chosen finite difference scheme and $\mathbf{b}^n$ contains values from the previous time step.

4 Numerical Methods

4.1 Projected Successive Over-Relaxation (PSOR)

Seydel discusses the PSOR method as a standard iterative method approach for solving the discrete LCP. The algorithm proceeds as follows:

1. Initialize $\mathbf{V}^{n,0} = \mathbf{V}^{n-1}$
2. For each iteration k and each grid point i:

$$V_i^{n,k} = \max\left(\Lambda_i, V_i^{n,k-1} + \omega \frac{b_i^n - \sum_{j<i} A_{ij} V_j^{n,k} - \sum_{j\geq i} A_{ij} V_j^{n,k-1}}{A_{ii}}\right)$$

3. Repeat until convergence: $\|\mathbf{V}^{n,k} - \mathbf{V}^{n,k-1}\| < \epsilon$

Here, $\omega \in (1, 2)$ is the relaxation parameter which can accelerates convergence. PSOR is robust and simple but can be computationally expensive due to iterative nature at each time step.

4.2 Operator Splitting Method

Ikonen and Toivanen (2004) proposed a more efficient approach based on operator splitting, which decouples the PDE solution from the early-exercise constraint. The method can also be interpreted using a Lagrange multiplier formulation.

4.2.1 Mathematical Formulation

The PDE with Lagrange multiplier is:

$$\frac{\partial V}{\partial \tau} - \mathcal{L}_{BS} V - \lambda = 0, \quad \lambda \geq 0, \quad V \geq \Lambda, \quad \lambda(V - \Lambda) = 0, \tag{10}$$

where $V(S, \tau)$ is the option value, $\tau = T - t$ is backward time, $\mathcal{L}_{BS}$ is the Black–Scholes operator, $\Lambda(S)$ is the intrinsic value, and λ is a Lagrange multiplier enforcing $V \geq \Lambda$.

4.2.2 Algorithm

At each time step, the operator splitting method consists of two stages:

1. **Evolution Step:** Solve the Black-Scholes PDE without constraints using a standard implicit, explicit or Crank-Nicolson scheme:

$$(I - \Delta t A)\tilde{\mathbf{V}}^{n+1} = \mathbf{V}^n, \tag{11}$$

   where $\tilde{\mathbf{V}}^{n+1}$ is the temporary solution vector, $\mathbf{V}^n$ is the option value at the previous time step, A is the discretization matrix for $\mathcal{L}_{BS}$, I is the identity matrix, and Δt is the time step size.

2. **Projection Step:** Enforce the early-exercise condition:

$$\mathbf{V}^{n+1} = \max(\tilde{\mathbf{V}}^{n+1}, \mathbf{\Lambda}), \tag{12}$$

   where $\mathbf{V}^{n+1}$ is the option value vector after applying the constraint and $\mathbf{\Lambda}$ is the vector of intrinsic values at each grid point.

This splitting technique decouples the PDE solution from the constraint enforcement, transforming an iterative LCP problem into sequential direct computations. The resulting algorithm is both computationally much less expensive and faster than PSOR.

5 Implementation

Both methods are implemented in Python as presented in articel with the following parameters; spot price: $S_0 = 14$, Strike: $K = 15$, Risk-free rate: $r = 0.1$, Volatility: $\sigma = 0.25$, Maturity: $T = 0.5$ years, Asset grid: $M = 200$, Time grid: $N = 500$ and Maximum asset price: $S_{\max} = 50$.

6 Results

The evolution of the option price at the spot price $S_0 = 14$ over time shows that both methods converge smoothly to maturity. While the option price difference between the two methods is small, the computation time differs significantly, as illustrated in Figure 1.

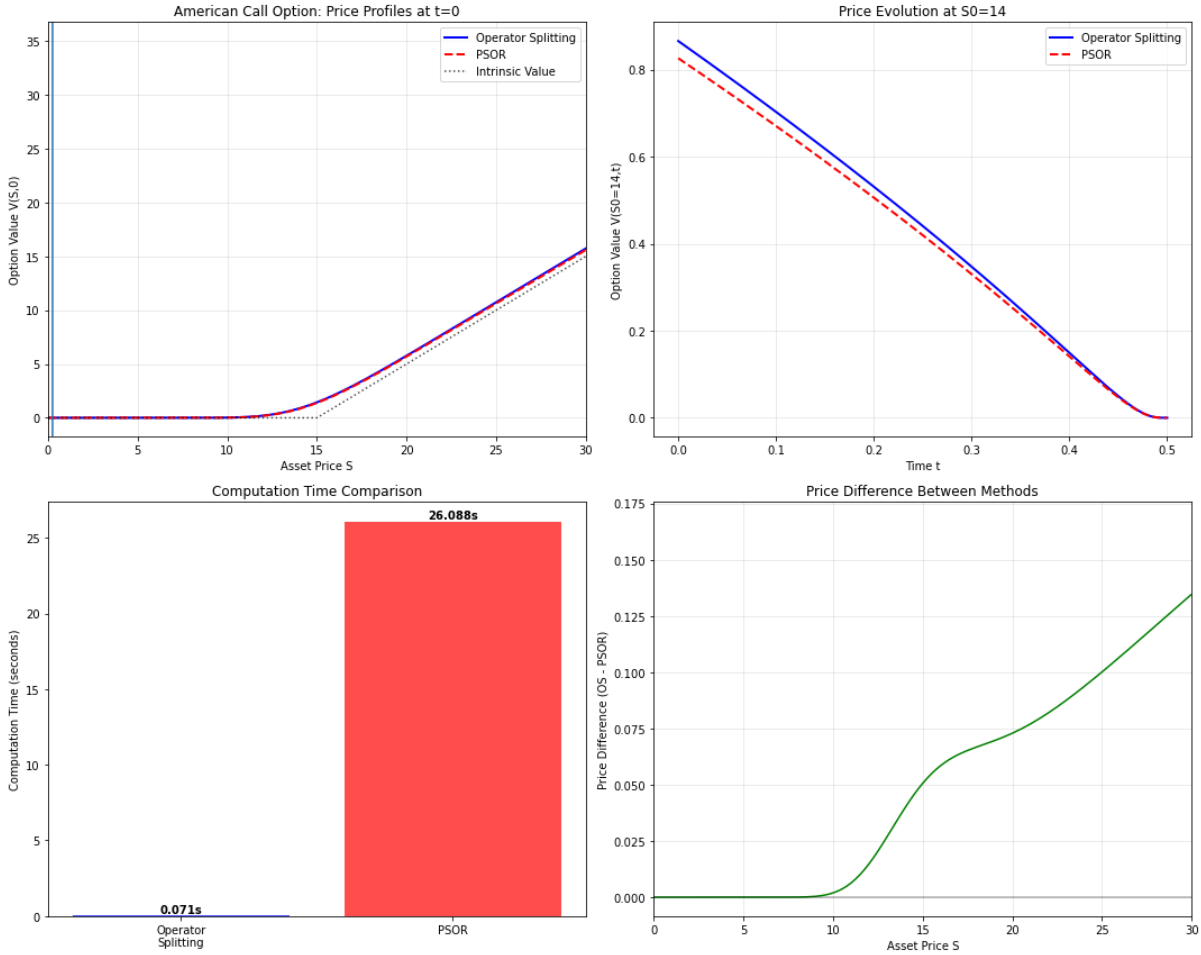

Figure 1: Option price profiles using PSOR (red dashed) and Operator Splitting (blue solid). Intrinsic value is shown in black dotted.

7 Comparision and Discussion

Both methods are based on finite difference discretization and complementarity conditions but differ in computational efficiency which is evident from the figure above. The PSOR approach directly solves the LCP, guaranteeing convergence but at the cost of higher computational time. The operator splitting method, by contrast, decouples the constraint handling, allowing for simpler linear solves and faster execution.

Numerical experiments in Ikonen and Toivanen (2004) demonstrate that the operator splitting approach achieves similar accuracy to PSOR while significantly reducing computation time. It is also well-suited for parallelization and higher-dimensional problems.

8 Conclusion

Finite difference methods combined with LCP formulation allowed accurate and efficient pricing of American options. PSOR provides a classical approach, while operator splitting offers computational advantages. The operator splitting technique of Ikonen and Toivanen represents an important advancement, offering comparable accuracy with improved computational efficiency.

9 References

- Seydel, R. *Tools for Computational Finance*, Springer, 2009.
- Ikonen, S., Toivanen, J. "Operator splitting methods for American options", *IMA Journal of Numerical Analysis*, 2004.

Appendix

In this appendix, we provide the Python code used to implement the numerical methods and generate the figures presented in the report.

```


import numpy as np
import matplotlib.pyplot as plt
import scipy.linalg as la
import time

class AmericanOptionPricer:
    def __init__(self, S0=14, K=15, r=0.1, sigma=0.25, T=0.5, M=200, N=500, S_max=50):
        # Option parameters
        self.S0 = S0
        self.K = K
        self.T = T
        self.r = r
        self.sigma = sigma
        # Grid parameters
        self.M = M
        self.N = N
        self.S_max = S_max
        self.dS = S_max / M
        self.dt = T / N
        # Grids
        self.S = np.linspace(0, S_max, M+1)
        self.t = np.linspace(0, T, N+1)
        self.intrinsic = np.maximum(self.S - K, 0)  # CALL option
        self._setup_matrix()

    def _setup_matrix(self):
        M, dt, r, sigma = self.M, self.dt, self.r, self.sigma
        alpha = np.zeros(M+1)
        beta = np.zeros(M+1)
        gamma = np.zeros(M+1)

        for i in range(1, M):
            alpha[i] = 0.25 * dt * (sigma**2 * self.S[i]**2 / self.dS**2 - r*self.S[i]/self.dS)
            beta[i] = -0.5 * dt * (sigma**2 * self.S[i]**2 / self.dS**2 + r)
            gamma[i] = 0.25 * dt * (sigma**2 * self.S[i]**2 / self.dS**2 + r*self.S[i]/self.dS)

        # LHS matrix (I - 0.5*dt*A) for Crank-Nicolson
        self.A = np.zeros((M+1, M+1))
```

```python
        for i in range(1, M):
            self.A[i, i-1] = -alpha[i]
            self.A[i, i] = 1 - beta[i]
            self.A[i, i+1] = -gamma[i]
        self.A[0, 0] = self.A[M, M] = 1

        # RHS matrix (I + 0.5*dt*A) for Crank-Nicolson
        self.B = np.zeros((M+1, M+1))
        for i in range(1, M):
            self.B[i, i-1] = alpha[i]
            self.B[i, i] = 1 + beta[i]
            self.B[i, i+1] = gamma[i]
        self.B[0, 0] = self.B[M, M] = 1

    def price_psor(self, omega=1.5, tol=1e-6, max_iter=1000):
        """PSOR method as described in Seydel"""
        V = self.intrinsic.copy()
        price_evolution = []
        iterations_per_step = []

        start = time.time()
        price_evolution.append(np.interp(self.S0, self.S, V))

        for n in range(1, self.N + 1):
            b = self.B @ V
            V_old = V.copy()
            error = tol + 1
            iters = 0

            while error > tol and iters < max_iter:
                error = 0
                for i in range(1, self.M):
                    gs_update = (b[i] - np.dot(self.A[i, :i], V[:i]) -
                                 np.dot(self.A[i, i+1:], V_old[i+1:])) / self.A[i, i]
                    new_val = V_old[i] + omega * (gs_update - V_old[i])
                    new_val = max(self.intrinsic[i], new_val)
                    error += abs(new_val - V_old[i])
                    V_old[i] = new_val
                iters += 1

            iterations_per_step.append(iters)
            V = V_old
            price_evolution.append(np.interp(self.S0, self.S, V))

        price_evolution_forward = price_evolution[::-1]
        comp_time = time.time() - start

        return {
            'price_vector': V,
            'price_time': price_evolution_forward,
            'computation_time': comp_time,
            'iterations': iterations_per_step
        }

    def price_operator_splitting(self, implement_lagrange_form=False):
        """Operator splitting method as described in Ikonen & Toivanen"""
```

```python
        V = self.intrinsic.copy()
        price_evolution = []

        start = time.time()
        price_evolution.append(np.interp(self.S0, self.S, V))

        # Precompute LU decomposition for efficiency
        A_lu = la.lu_factor(self.A)

        for n in range(1, self.N + 1):
            b = self.B @ V
            y = la.lu_solve(A_lu, b)

            if implement_lagrange_form:
                # Lagrange multiplier formulation with corrected sign
                # Equation (10) from the article with sign correction
                lambda_val = np.maximum(0, (self.intrinsic - y) / self.dt)
                V = y + self.dt * lambda_val  # Corrected sign
            else:
                # Equivalent projection form (computational)
                V = np.maximum(y, self.intrinsic)

            price_evolution.append(np.interp(self.S0, self.S, V))

        price_evolution_forward = price_evolution[::-1]
        comp_time = time.time() - start

        return {
            'price_vector': V,
            'price_time': price_evolution_forward,
            'computation_time': comp_time
        }



# ----------------- Comprehensive Comparison -----------------
def run_comprehensive_comparison():
    print("=== COMPREHENSIVE COMPARISON: PSOR vs OPERATOR SPLITTING ===")
    print(f"Parameters: S0=14, K=15, r=0.1,   =0.25, T=0.5")
    print("=" * 60)

    pricer = AmericanOptionPricer(S0=14, K=15, r=0.1, sigma=0.25, T=0.5, M=200, N=500)

    print("\n1. OPERATOR SPLITTING METHOD")
    results_os = pricer.price_operator_splitting(implement_lagrange_form=True)

    print(f"   Price at S0: {results_os['price_time'][-1]:.6f}")
    print(f"   Computation time: {results_os['computation_time']:.4f}s")


    print("\n2. PSOR METHOD")
    results_psor = pricer.price_psor()

```

```python
    print(f"   Price at S0: {results_psor['price_time'][-1]:.6f}")
    print(f"   Computation time: {results_psor['computation_time']:.4f}s")

    print(f"   Avg iterations/step: {np.mean(results_psor['iterations']):.1f}")

    print("\n3. COMPARISON METRICS")
    speedup = results_psor['computation_time'] / results_os['computation_time']
    price_diff = abs(results_os['price_time'][-1] - results_psor['price_time'][-1])


    print(f"   Speedup factor: {speedup:.2f}x")
    print(f"   Price difference: {price_diff:.6f}")



    return pricer, results_os, results_psor

def create_comprehensive_plots(pricer, results_os, results_psor):
    """Create plots that match the report's analysis"""
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))

    # Plot 1: Option price profiles at t=0
    axes[0, 0].plot(pricer.S, results_os['price_vector'], 'b-', linewidth=2,
                    label='Operator Splitting')
    axes[0, 0].plot(pricer.S, results_psor['price_vector'], 'r--', linewidth=2,
                    label='PSOR')
    axes[0, 0].plot(pricer.S, pricer.intrinsic, 'k:', alpha=0.7,
                    label='Intrinsic Value')


    axes[0, 0].set_xlabel('Asset Price S')
    axes[0, 0].set_ylabel('Option Value V(S,0)')
    axes[0, 0].set_title('American Call Option: Price Profiles at t=0')
    axes[0, 0].legend()
    axes[0, 0].grid(True, alpha=0.3)
    axes[0, 0].set_xlim(0, 30)

    # Plot 2: Price evolution at S0=14
    axes[0, 1].plot(pricer.t, results_os['price_time'], 'b-', linewidth=2,
                    label='Operator Splitting')
    axes[0, 1].plot(pricer.t, results_psor['price_time'], 'r--', linewidth=2,
                    label='PSOR')
    axes[0, 1].set_xlabel('Time t')
    axes[0, 1].set_ylabel(f'Option Value V(S0=14,t)')
    axes[0, 1].set_title('Price Evolution at S0=14')
    axes[0, 1].legend()
    axes[0, 1].grid(True, alpha=0.3)

    # Plot 3: Computation time comparison
    methods = ['Operator\nSplitting', 'PSOR']
```

```python
    times = [results_os['computation_time'], results_psor['computation_time']]
    bars = axes[1, 0].bar(methods, times, color=['blue', 'red'], alpha=0.7)
    axes[1, 0].set_ylabel('Computation Time (seconds)')
    axes[1, 0].set_title('Computation Time Comparison')
    for bar in bars:
        height = bar.get_height()
        axes[1, 0].text(bar.get_x() + bar.get_width()/2., height,
                        f'{height:.3f}s', ha='center', va='bottom', fontweight='bold')

    # Plot 4: Price difference
    price_diff = results_os['price_vector'] - results_psor['price_vector']
    axes[1, 1].plot(pricer.S, price_diff, 'g-', linewidth=1.5)
    axes[1, 1].axhline(y=0, color='k', linestyle='-', alpha=0.3)
    axes[1, 1].set_xlabel('Asset Price S')
    axes[1, 1].set_ylabel('Price Difference (OS - PSOR)')
    axes[1, 1].set_title('Price Difference Between Methods')
    axes[1, 1].grid(True, alpha=0.3)
    axes[1, 1].set_xlim(0, 30)

    plt.tight_layout()
    plt.savefig('comprehensive_comparison_report.png', dpi=300, bbox_inches='tight')
    plt.show()

if __name__ == "__main__":
    pricer, results_os, results_psor = run_comprehensive_comparison()
    create_comprehensive_plots(pricer, results_os, results_psor)
```

Listing 1: Python code for the project